

Network Intrusion Detection & Prevention Systems Exploitation

Use of Suricata IPS

(Part 2)

I. Suricata IPS configuration

In this part of the Lab, we are going to configure Suricata as an inline IPS. For that, we need to configure the installed firewall (iptables) to forward all the traffic to Suricata to be treated before reaching the supervised network. Using this IPS, we will block any malicious traffic [1].

NFQUEUE is by suricata when working as an IPS. With NFQUEUE it is possible to delegate the verdict on the packet to a userspace software. The Linux kernel will ask a userspace software connected to a queue for a decision.

With NFQUEUE incoming packet matched by an iptables rule is sent to Suricata through nfnetlink. Suricata receives the packet and issues a verdict depending on our ruleset. The packet is either transmitted or rejected by kernel. Consequently, the action NFQUEUE will be used, instead of ACCEPT for the iptables rules. Using NFQUEUE, iptables will send the packets for separate and specific queue that will be used as an input for Suricata.

1. The interaction between iptables and Suricata will be configured in the section “nfq” of the file.

`/etc/suricata/suricata.yaml` as follows :

```
nfq:
mode: accept #By default the packet will be accepted or dropped by Suricata
#repeat_mark: 1 #If the mode is set to 'repeat', the packets will be marked after being
processed by Suricata.
#repeat_mask: 1
#route_queue: 2 #Here you can assign the queue-number of the tool that Suricata has to
send the packets #to after processing
them.
```

- If the mode is set to 'accept', the packet that has been send to Suricata by a rule using NFQ, will not be inspected by the rest of the iptables rules after being processed by Suricata.
- If the mode is set to 'repeat', the packets will be marked by Suricata and then re-injected at the first rule of iptables. To mitigate the packets from being going round in circles, the rule using NFQ will be skipped because of the mark.
- If the mode is set to 'route', the packet will be sent to another tool after being processed by Suricata.

It is possible to assign this tool at the mandatory option 'route_queue' and link every engine/tool to a different queue-number.

```
GNU nano 2.9.8 /etc/suricata/suricata.yaml Modified
nfq:
  mode: accept
# repeat-mark: 1
# repeat-mask: 1
# bypass-mark: 1
# bypass-mask: 1
# route-queue: 2
# batchcount: 20
# fail-open: yes
```

2. Check if NFQ is enabled in Suricata.

For this purpose, use the following command:

```
# sudo suricata --build-info
```

```
[centos@centosstream8 ~]$ sudo suricata --build-info
This is Suricata version 6.0.20 RELEASE
Features: NFQ PCAP_SET_BUFF AF_PACKET HAVE_PACKET_FANOUT LIBCAP_NG LIBNET1.1 HAVE_HTTP_URI_NORMALIZE_HOOK PCRE_JIT HAVE_NSS HAVE_LUA HAVE_LIBJANSSON TLS TLS_C11 MAGIC_RUST
SIMD support: none
Atomic intrinsics: 1 2 4 8 byte(s)
64-bits, Little-endian architecture
GCC version 8.5.0 20210514 (Red Hat 8.5.0-22), C version 201112
compiled with _FORTIFY_SOURCE=2
L1 cache line size (CLS)=64
thread local storage method: _Thread_local
compiled with LibHTTP v0.5.48, linked against LibHTTP v0.5.48

Suricata Configuration:
  AF_PACKET support:          yes
  eBPF support:               no
  XDP support:                no
  PF_RING support:            no
  NFQueue support:            yes
  NFlOG support:              no
  IPFW support:               no
  Netmap support:             no   using new api: no
  DAG enabled:                no
  Napatech enabled:           no
  WinDivert enabled:          no
```

3. Check the default rules and the default policy for the chains INPUT and OUTPUT

```
# sudo iptables -L
```

```
[centos@centosstream8 ~]$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
LIBVIRT_INP all  --  anywhere              anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination
LIBVIRT_FWX all  --  anywhere              anywhere
LIBVIRT_FWI all  --  anywhere              anywhere
LIBVIRT_FWO all  --  anywhere              anywhere

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
LIBVIRT_OUT all  --  anywhere              anywhere
```

4. To send all the incoming and outgoing traffic from iptables to Suricata

Use the following commands:

```
# iptables -F
# iptables -A OUTPUT -j NFQUEUE
# iptables -A INPUT -j NFQUEUE
```

```
[centos@centosstream8 ~]$ sudo iptables -F
[centos@centosstream8 ~]$ sudo iptables -A OUTPUT -j NFQUEUE
[centos@centosstream8 ~]$ sudo iptables -A INPUT -j NFQUEUE
```

5. Open the Suricata configuration file `/etc/suricata/suricata.yaml` in your editor and scroll down to the "stream" section.

There, set "inline" to "yes". This will force Suricata to do stream reassembly in an IPS aware way.

```
GNU nano 2.9.8 /etc/suricata/suricata.yaml Modified
# # check if a segment contains different data
# # than what we've already seen for that
# # position in the stream.
# # This is enabled automatically if inline mode
# # is used or when stream-event:reassembly_overl$
# # is used in a rule.
#
stream:
  memcap: 64mb
  #memcap-policy: ignore
  checksum-validation: yes # reject incorrect csums
  #midstream: false
  #midstream-policy: ignore
  inline: yes # auto will use inline mode in IPS mode, yes or $
```

II. Preventing against ICMP echo requests

6. Filtering ICMP echo requests can help conserve network resources by reducing unnecessary traffic.

This is especially relevant in environments where bandwidth is limited.
Execute In `/var/lib/suricata/rules/custom.rules` change the icmp detection rule as follows to block any icmp ping request datagram sent to server 8.8.8.8

```
# vi /var/lib/suricata/rules/custom.rules
```

```
GNU nano 2.9.8 /var/lib/suricata/rules/custom.rules

drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0; itype:8; msg: "Ping Detected &$
```

7. To run suricata as an IPS with the NFQ mode, you must make use of the -q option.

This option tells Suricata which of the queue numbers it should use.

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

```
[centos@centosstream8 ~]$ sudo suricata -c /etc/suricata/suricata.yaml -q 0
22/10/2025 -- 01:14:52 - <Notice> - This is Suricata version 6.0.20 RELEASE running in SYSTEM mode
22/10/2025 -- 01:16:02 - <Notice> - all 4 packet processing threads, 4 management threads initialized, engine started.
```

8. the following commands to test Suricata response.

```
# ping 8.8.8.8
```

```
[centos@centosstream8 ~]$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
23 packets transmitted, 0 received, 100% packet loss, time 22555ms
```

```
# nslookup http://portquiz.net/
```

```
# nslookup portquiz.net
```

```
[centos@centosstream8 ~]$ nslookup portquiz.net
Server:      192.168.43.1
Address:     192.168.43.1#53

Non-authoritative answer:
Name:   portquiz.net
Address: 35.180.139.74
Name:   portquiz.net
Address: 64:ff9b::23b4:8b4a
```

```
# curl portquiz.net
```

```
[centos@centosstream8 ~]$ curl portquiz.net  
Port test successful!  
Your IP: 197.27.245.13
```

9. There should be no response to the ping command, while DNS and HTTP connection should be possible.

Check the log file fast.log and verify that suricata has successfully blocked the ICMP traffic, logged the event and generated the alert.

```
# tail -f /var/log/suricata/fast.log
```

```
10/22/2025-02:06:17.395745 [Drop] [**] [1:1000001:1] Ping Detected & Blocked [*  
*] [Classification: Generic ICMP event] [Priority: 3] {ICMP} 10.0.2.15:8 -> 8.8.  
8.8:0
```

III. Filtering clear HTTP traffic

Many data protection regulations and standards, such as GDPR, require organizations to implement measures to protect the confidentiality of sensitive data. Filtering clear HTTP traffic is one way to enhance data protection and align with regulatory requirements.

10. In `/var/lib/suricata/rules/custom.rules` add the following rule to block any access to HTTP websites

```
# vi /var/lib/suricata/rules/custom.rules
```

```
drop tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"Dropped HTTP traffic"; priority:1;sid:1000002; rev:1;)
```

```
GNU nano 2.9.8 /var/lib/suricata/rules/custom.rules
drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0;itype:8;msg:"Ping Detected & Bl$
drop tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"Dropped HTTP traffic"$
```

11. Stop suricata and start it again as an IPS with the NFQ mode

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

12. Using the command `curl` try to access to `http://portquiz.net/` through two different TCP ports 80 and 445 (usually used by SMB protocol).

Note that the HTTP server portquiz.net is especially listening to all TCP ports, allowing HTTP access through anyone of them. Argue the results that you obtain

```
[centos@centosstream8 ~]$ curl http://www.portquiz.net
^C
[centos@centosstream8 ~]$ curl http://www.portquiz.net:445
Port test successful!
Your IP: 197.27.245.13
[centos@centosstream8 ~]$ curl http://www.portquiz.net:80
^C
10/22/2025-02:30:23.294649 [Drop] [**] [1:1000002:1] Dropped HTTP traffic [**]
[Classification: (null)] [Priority: 1] {TCP} 10.0.2.15:41746 -> 35.180.139.74:80
10/22/2025-02:30:33.561224 [**] [1:2210054:1] SURICATA STREAM excessive retransmissions [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 216.58.204.234:443 -> 10.0.2.15:43402
```

13. To prevent access to portquiz website through any possible port, we propose to update the previous rule as follows:

```
drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"Dropped HTTP traffic";
content:"User-Agent: "; http_header; nocase; priority:1; sid:1000002; rev:1;)
```

```
GNU nano 2.9.8 /var/lib/suricata/rules/custom.rules
drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0;itype:8;msg:"Ping Detected & Bl$
drop tcp $HOME_NET any -> $EXTERNAL_NET any(msg:"Dropped HTTP traffic"; contents$
```

14. Access again to portquiz website through the two TCP ports 80 and 445 using the browser Firefox.

Now, the website is completely unreachable. Explain why?

http://portquiz.net

http://portquiz.net:445

```
[centos@centosstream8 ~]$ curl http://www.portquiz.net
^C
```

```
[centos@centosstream8 ~]$ curl http://www.portquiz.net:445
^C
```

15. Check the log file fast.log, and verify that suricata has successfully blocked the two HTTP connections (port 80 and port 445)

```
10/22/2025-03:44:03.601314 [Drop] [**] [1:1000003:1] Dropped HTTP/HTTPS/SMB tra
ffic [**] [Classification: (null)] [Priority: 1] {TCP} 10.0.2.15:58992 -> 35.180
.139.74:445
```

```
10/22/2025-03:44:36.342618 [Drop] [**] [1:1000003:1] Dropped HTTP/HTTPS/SMB tra
ffic [**] [Classification: (null)] [Priority: 1] {TCP} 10.0.2.15:41984 -> 35.180
.139.74:80
```

16. Using your browser Firefox try to access the website <https://www.google.com> Is access possible? Explain why?

```
10/22/2025-03:57:18.403229 [Drop] [**] [1:1000003:1] Dropped HTTP/HTTPS/SMB tra
ffic [**] [Classification: (null)] [Priority: 1] {TCP} 10.0.2.15:51376 -> 142.25
1.209.36:443
```

- Suricata is running in Intrusion Prevention System (IPS) mode
- The custom rules are actively blocking network traffic
- Log entries showing "[Drop]" provide evidence of the blocking action

IV. TLS analysis

17. Connect to the website <https://badssl.com/> and click on “expired” to access the subdomain <https://expired.badssl.com/> that uses an expired certificate.

You should click on accept the risk and continue if your browser asks for confirmation. Check that the validity period of the certificate has ended.

18. In the rule file `custom.rules` add the following entry to drop access to sites with expired certificates

```
drop tls any any -> any any (msg:"expired certs"; tls_cert_expired; priority:1; sid:1000005; rev:1;)
```

```
GNU nano 2.9.8 /var/lib/suricata/rules/custom.rules
drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0;itype:8;msg:"Ping Detected & Bl$
drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"Dropped HTTP traffic"; conten$
# drop tcp $HOME_NET any -> $EXTERNAL_NET [445,443,80] (msg:"Dropped HTTP/HTTPS$
drop tls any any -> any any (msg:"expired certs"; tls_cert_expired; priority:1;$
```

19. Stop suricata and start it again as an IPS with the NFQ mode

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

20. Test that access to <https://expired.badssl.com/> is dropped and that the alert is generated.

```
[centos@centosstream8 ~]$ curl https://expired.badssl.com/
^C
10/22/2025-04:20:04.649068 [Drop] [**] [1:1000005:1] expired certs [**] [Classi
fication: (null)] [Priority: 1] {TCP} 104.154.89.105:443 -> 10.0.2.15:46874
```


21. In the rule file custom.rules add a new entry to drop access to facebook website using TLS, matching the content of facebook X509 certificate.

→ `drop tls any any -> any any (msg:"Facebook TLS Certificate Blocked";
tls.cert_subject; content:"facebook.com"; nocase; sid:1000006; rev:1;)`

```
GNU nano 2.9.8 /var/lib/suricata/rules/custom.rules  
  
drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0;itype:8;msg:"Ping Detect$  
drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"Dropped HTTP traffic";$  
  
# drop tcp $HOME_NET any -> $EXTERNAL_NET [445,443,80] (msg:"Dropped HTT$  
  
drop tls any any -> any any (msg:"expired certs"; tls_cert_expired; prio$  
drop tls any any -> any any (msg:"facebook TLS certificate Blocked"; tls$
```

22. Now open Firefox, and try to go to <http://www.facebook.com/>.

The request should time out.

23. Check the log file `/var/log/suricata/fast.log` and verify the displayed alert message

V. Downloaded files collection

24. Open `suricata.yaml` configuration file, go to the output section, and update the file-store variable to enable file extraction capability.

```
GNU nano 2.9.8 /etc/suricata/suricata.yaml Modified
# record. A fileinfo file will be written for each occurrence of the
# file seen using a filename suffix to ensure uniqueness.
#
# To prune the filestore directory see the "suricatactl filestore
# prune" command which can delete files over a certain age.
- file-store:
  version: 2
  enabled: yes
```

25. In `/var/lib/suricata/rules/custom.rules` add the following rule to store all files to disk.

alert http any any -> any any (msg:"FILE store all"; filestore; sid:1; rev:1;)

```
GNU nano 2.9.8 /var/lib/suricata/rules/custom.rules
drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0;itype:8;msg:"Ping Detect$
drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"Dropped HTTP traffic";$
# drop tcp $HOME_NET any -> $EXTERNAL_NET [445,443,80] (msg:"Dropped HTT$
drop tls any any -> any any (msg:"expired certs"; tls_cert_expired; prio$
drop tls any any -> any any (msg:"facebook TLS certificate Blocked"; tls$
alert http any any -> any any (msg:"FILE store all"; filestore; sid:1; r$
```

26. Stop and reexecute suricata using the command:

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

27. Download a GIF image through a plaintext HTTP connection

28. Go to the folder `Review the folders/var/log/suricata/filestore/` and execute the following commands.

You should notice that one of the stored images is a GIF file.

```
# cd /var/log/suricata/filestore/
# file */*
```

29. Open the stored file using the command display (you should install ImageMagick using yum if it is not already available in your system) and check the obtained image.

```
# display 4b/4bfe3adb36911561eefe2421a674465ab14a9f42879a986f71bc844612fa9981
```

VI. Generating and analyzing alerts in JSON format

30. Suricata can output alerts, http events, dns events, tls events and file info through EVE json format. The JSON format allows a nice handling of data in external tools like Elasticsearch or Splunk.

Open the Suricata configuration file `/etc/suricata/suricata.yaml` in your editor and scroll down to the "output" section and check that logging in eve-log format is enabled:

```
GNU nano 2.9.8 /etc/suricata/suricata.yaml

# Extensible Event Format (nicknamed EVE) event log in JSON format
- eve-log:
  enabled: yes
  filetype: regular #regular|syslog|unix_dgram|unix_stream|redis
  filename: eve.json
```

31. Download the PCAP traffic capture of the WANNACRY ransomware attack which spreads using ETERNALBLUE exploit, and then unzip the pcap file (the password is: infected) using the two commands:

```
# wget
https://raw.githubusercontent.com/tatsuiman/malware-traffic-analysis.net/a5a261cacaf84079e4c0e0f2729c81226a27eca7/2017-05-18-WannaCry-ransomware-using-ExternalBlue-exploit.pcap
```

```
[centos@centosstream8 ~]$ wget https://raw.githubusercontent.com/tatsuiman/malware-traffic-analysis.net/a5a261cacaf84079e4c0e0f2729c81226a27eca7/2017-05-18-WannaCry-ransomware-using-ExternalBlue-exploit.pcap
--2025-10-22 05:03:11-- https://raw.githubusercontent.com/tatsuiman/malware-traffic-analysis.net/a5a261cacaf84079e4c0e0f2729c81226a27eca7/2017-05-18-WannaCry-ransomware-using-ExternalBlue-exploit.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.111.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
```

```
Length: 37791327 (36M) [application/octet-stream]
Saving to: '2017-05-18-WannaCry-ransomware-using-ExternalBlue-exploit.pcap'

2017-05-18-WannaCr 100%[=====>] 36.04M 243KB/s in 2m 30s

2025-10-22 05:05:43 (246 KB/s) - '2017-05-18-WannaCry-ransomware-using-ExternalBlue-exploit.pcap' saved [37791327/37791327]
```

32. Make sure that the rule file `suricata.rules` is activated in the configuration file `/etc/suricata/suricata.yaml`

```
GNU nano 2.9.8 /etc/suricata/suricata.yaml

##

default-rule-path: /var/lib/suricata/rules

rule-files:
- suricata.rules
- custom.rules
```

33. Read the pcap file using suricata, while disabling all checksum checking in the loaded datagrams

```
# suricata -r 2017-05-18-WannaCry-ransomware-using-EnternalBlue-exploit.pcap -c
/etc/suricata/suricata.yaml -l /var/log/suricata/ -k none
```

```
[centos@centosstream8 ~]$ sudo suricata -r 2017-05-18-WannaCry-ransomware-using-EnternalBlue-exploit.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata/ -k none
24/10/2025 -- 02:02:18 - <Notice> - This is Suricata version 6.0.20 RELEASE running in USER mode
24/10/2025 -- 02:02:51 - <Notice> - all 4 packet processing threads, 4 management threads initialized, engine started.
24/10/2025 -- 02:02:51 - <Notice> - Signal Received. Stopping engine.
24/10/2025 -- 02:02:51 - <Notice> - Pcap-file module read 1 files, 46654 packets, 37044839 bytes
```

Open the content of `fast.log` and read it. You can also search for keyword `exploit` in `fast.log` using the command:

```
# grep -in exploit /var/log/suricata/fast.log
```

```
[centos@centosstream8 ~]$ sudo grep -in exploit /var/log/suricata/fast.log
145:05/18/2017-13:42:21.155613  [**] [1:2024297:2] ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010 [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.116.149:49608 -> 192.168.116.172:445
148:05/18/2017-13:42:27.488303  [**] [1:2024297:2] ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010 [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.116.149:49767 -> 192.168.116.143:445
149:05/18/2017-13:42:27.963712  [**] [1:2024297:2] ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010 [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.116.149:49767 -> 192.168.116.143:445
150:05/18/2017-13:42:31.317822  [**] [1:2024297:2] ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010 [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.116.149:49767 -> 192.168.116.143:445
```

34. While the json format is readable by human but as an event/record can contain a lot of data it can be difficult to do a by-eye analysis when looking at a file.

Therefore, we will use the jq utility which is a tool dedicated to the transformation/parsing of a JSON entry. You can review the json formatted version of the log available in:

<https://suricata.readthedocs.io/en/suricata-6.0.2/output/eve/eve-json-format.html>

35. Review all alerts using the command

```
# cd /var/log/suricata/
```

```
# jq . eve.json | less
```

```
[root@centosstream8 ~]# cd /var/log/suricata/
[root@centosstream8 suricata]# jq . eve.json | less
```

```
{
  "timestamp": "2025-10-13T16:18:43.499980+0530",
  "event_type": "stats",
  "stats": {
    "uptime": 49,
    "capture": {
      "kernel_packets": 0,
      "kernel_drops": 0,
      "errors": 0
    },
    "decoder": {
      "pkts": 0,
      "bytes": 0,
      "invalid": 0,
      "ipv4": 0,
      "ipv6": 0,
      "ethernet": 0,
      "chdlc": 0,
      "raw": 0,
      "null": 0,
      "sll": 0,
      "tcp": 0,
      "udp": 0,
      "sctp": 0,
      "icmpv4": 0,
      "icmpv6": 0,
      "ppp": 0,
      "pppoe": 0,
      "geneve": 0,
      "gre": 0,
      "vlan": 0,
      "vlan_qinq": 0,
      "vxlan": 0,
      "vntag": 0,
      "ieee8021ah": 0,
      "teredo": 0,
```

36. To find alerts with a severity of 1 and display the source/destination IP and the alert, execute the following command:

```
# jq -c 'select(.alert.severity==1) | [.src_ip,.dest_ip,
.alert.signature]' /var/log/suricata/eve.json | less
```

You can also group each alert, and display the highest count at the top of the list, using the command:

```
# jq -c 'select(.alert.severity==1) | [.src_ip,.dest_ip,
.alert.signature]' /var/log/suricata/eve.json | sort | uniq -c | sort -rn | more
```

```
[root@centosstream8 suricata]# jq -c 'select(.alert.severity==1) | [.src_ip,.dest_ip,.alert.signature]' /var/log/suricata/eve.json | sort | uniq -c | sort -rn | more
 22 ["192.168.116.149","192.168.116.143","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
 12 ["192.168.116.149","192.168.116.172","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
  8 ["192.168.116.149","192.168.116.138","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
  8 ["192.168.116.138","192.168.116.149","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
  2 ["192.168.116.138","192.168.116.172","ET EXPLOIT ETERNALBLUE Exploit M2 MS17-010"]
  2 ["10.0.2.15","57.144.120.1","Blocked Facebook TLS"]
  2 ["10.0.2.15","142.251.37.196","ET USER_AGENTS Suspicious User Agent (BlackSun)"]
  1 ["104.154.89.105","10.0.2.15","expired certs"]
  1 ["10.0.2.15","35.180.139.74","Dropped HTTP traffic"]
```

To view all the alerts and show the IP address and port information, execute the following command:

```
# grep '"event_type":"alert"' /var/log/suricata/eve.json | jq
'"(.timestamp) | \(.alert.gid):\(.alert.signature_id):\(.alert.rev) |
\(.alert.signature) | \(.alert.category) | \(.src_ip):\(.src_port) ->
\(.dest_ip):\(.dest_port)'"
```

```
[root@centosstream8 suricata]# grep '"event_type":"alert"' /var/log/suricata/eve.json | jq -r '"(.timestamp) | \(.alert.gid):\(.alert.signature_id):\(.alert.rev) | \(.alert.signature) | \(.alert.category) | \(.src_ip):\(.src_port) -> \(.dest_ip):\(.dest_port)'"
2025-10-13T16:33:49.406953+0530 | 1:2008983:8 | ET USER_AGENTS Suspicious User Agent (BlackSun) | A Network Trojan was detected | 10.0.2.15:55464 -> 142.251.37.196:80
2025-10-13T16:35:08.084029+0530 | 1:2008983:8 | ET USER_AGENTS Suspicious User Agent (BlackSun) | A Network Trojan was detected | 10.0.2.15:55466 -> 142.251.37.196:80
2025-10-20T14:54:49.034412+0530 | 1:1000002:0 | Dropped HTTP traffic | | 10.0.2.15:55998 -> 35.180.139.74:80
2025-10-20T15:38:24.835886+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33722 -> 34.107.221.82:80
2025-10-20T15:38:24.843282+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:37940 -> 81.171.33.202:80
2025-10-20T15:38:26.838965+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:56566 -> 2.16.70.4:80
2025-10-20T15:38:27.430208+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:56572 -> 2.16.70.4:80
2025-10-20T15:38:29.768964+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33764 -> 34.107.221.82:80
2025-10-20T15:38:31.048934+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:56064 -> 35.180.139.74:80
2025-10-20T15:38:34.780520+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33788 -> 34.107.221.82:80
2025-10-20T15:38:39.786836+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33790 -> 34.107.221.82:80
2025-10-20T15:38:43.862468+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:59444 -> 216.58.204.131:80
2025-10-20T15:38:44.787323+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33796 -> 34.107.221.82:80
2025-10-20T15:38:46.112290+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:59448 -> 216.58.204.131:80
2025-10-20T15:38:49.787781+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33800 -> 34.107.221.82:80
2025-10-20T15:39:24.760041+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33804 -> 34.107.221.82:80
2025-10-20T15:39:29.759191+0530 | 1:0:0 | Dropped HTTP traffic | | 10.0.2.15:33806 -> 34.107.221.82:80
```

37. Among the displayed alerts is the following:

"2017-05-18T13:42:07.220702+0530 | 1:2025992:2 | ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags) | A
Network Trojan was detected | 192.168.116.149:49368 -> 192.168.116.138:445"

Use the following command to determine the detection rule responsible for the alert generation

```
# grep -i "ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags)"  
/var/lib/suricata/rules/suricata.rules
```

```
[root@centosstream8 suricata]# grep -i "ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags)" /var/lib/suricata/rules/suricata.rules  
alert smb any any -> $HOME_NET any (msg:"ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags)"; flow  
:to_server,established; content:"|ff|SMB|25 00 00 00 00|"; offset:4; depth:9; content:"|00 00 00 00 00 00 00 00 00 00 00|"; distance:5; within:10; content:"|23 00 00 00 07 00 5c 50 49 50 45 5c 00|"; fast_pattern; endswith;  
threshold: type limit, track by_src, count 1, seconds 30; reference:url,github.com/rapid7/metasploit-framework/blob/master/modules/auxiliary/scanner/smb/smb_ms17_010.rb; classtype:trojan-activity; sid:2025992; rev:2; met  
adata:affected_product Windows_XP_Vista_7_8_10_Server_32_64_Bit, attack_target Client_Endpoint, created_at 2018_08_15, deployment Perimeter, malware_family ETERNALBLUE, confidence Medium, signature_severity Major, tag De  
scription_Generated_By_Proofpoint_Nexus, updated_at 2019_09_28;)
```